Mini Game Hub

Larissa
Sahil Deo

April 29, 2026

Contents

1 External Libraries **2**

2 Design & Architecture **2**
2.1 Authentication System (Bash) 2
2.2 Leaderboard System 2
2.3 Game Class Abstraction 2
2.4 Rendering and Scaling 3
2.5 Event Handling and Game Loop 3
2.6 Win Detection 3
2.7 Animations 3
2.8 Blitz Mode 3
2.9 User Interface Components 4
2.10 Othello Move Logic 4
2.11 Connect 4 Mechanics 4
2.12 Flowchart: 4

3 Features Implemented **4**

4 Directory Structure **5**

5 Installation & Setup **6**

6 Usage **6**

7 Retrospective **7**

8 What we would do differently with 10 more hours **8**

1 External Libraries

- **numpy** - Used for efficient handling of arrays and game board matrix. It simplifies operations on the game board, such as checking valid moves, updating states, and checking the win condition.
- **pygame** - Used for building the graphical user interface of the games. It handles rendering, mouse detection, game loops, and animations.
- **matplotlib** - Used for visualizing game statistics such as top 5 winners and the frequencies of each game
- **time** - Used for implementing delays, timers, and tracking time-based events within games.
- **os** - Used for interacting with the file system via bash (os.system())
- **sys** - Used for taking command line arguments for game.py

2 Design & Architecture

2.1 Authentication System (Bash)

User authentication is handled via shell scripts:

- User credentials are stored in `.users.tsv` with SHA-256 hashed passwords.
- Login allows 3 attempts before temporary lockout.
- Lockout timestamps are stored in `.failed_logins.csv`.

2.2 Leaderboard System

Game statistics are stored in CSV files. The leaderboard script:

- Sorts users by wins, losses, or win/loss ratio
- Formats output using `column`
- Maintains a top-5 players file

2.3 Game Class Abstraction

The `Game` class stores:

- The board as a NumPy 2D array
- Current player turn (boolean)
- Board dimensions and cell size

It provides utility functions such as `switchTurn()`, `valid_move()`, and a pluggable `checkWin()` function passed during initialization, allowing reuse across different games.

2.4 Rendering and Scaling

All rendering is done using `pygame`. Custom scaling functions `scale_h()` and `scale_w()` ensures resolution-independent scaling by mapping coordinates relative to a reference screen size.

Static elements such as boards are pre-rendered onto `Surface` objects and blitted every frame to improve performance.

2.5 Event Handling and Game Loop

Each game runs a main loop that:

- Captures user input (mouse clicks, key presses)
- Updates game state (moves, turn switching, timers)
- Renders updated visuals

Mouse clicks are mapped to board cells, and moves are executed only if valid.

2.6 Win Detection

Each game defines its own `checkWin()`:

- Tic Tac Toe and Connect 4 scan in 4 directions (horizontal, vertical, both diagonals) using NumPy slicing.
- Othello determines the winner by counting pieces when no valid moves remain (using `np.sum()`)

2.7 Animations

Animations are time-based using timestamps:

- Tic Tac Toe: progressive drawing of crosses (line segments) and circles (arc sweep).
- Connect 4: falling discs simulated using $y = \frac{1}{2}gt^2$.
- Othello: flipping discs using `pygame.draw.ellipse()` and expansion.

Jagged drawing effects are implemented via the `JaggedLine` class, which perturbs line segments randomly for a hand-drawn aesthetic. `pygame.draw.arc()` is naturally jagged, which is what inspired the hand-drawn aesthetic.

2.8 Blitz Mode

A `Timer` class manages turn-based countdowns. Each player's timer activates only during their turn. The remaining time is displayed as a dynamically coloured vertical bar, transitioning from green to red as time decreases.

2.9 User Interface Components

Reusable UI elements include:

- `Button`: handles hover, click feedback, and rendering
- `Box`: used for popups and text displays

End-of-game overlays use semi-transparent surfaces to dim the screen and display options like "Play Again" and "Main Menu".

2.10 Othello Move Logic

Valid moves are computed by scanning in all 8 directions from each empty cell. A move is valid if it captures at least one opponent piece bracketed by the current player's pieces. Piece flipping is performed by updating slices of the NumPy array.

2.11 Connect 4 Mechanics

Each column tracks the number of filled cells. When a move is made, the disc is placed in the lowest available row. Falling animation is applied until the disc reaches its final position.

2.12 Flowchart:

Start
↓
Authentication (main.sh)
↓
Select Game (game.py)
↓
Choose Time Control (game.py)
↓
Toss to Decide First Player (game.py)
↓

TicTacToe play()	Connect4 play()	Othello play()

↓
Return Result
↓
Leaderboard (leaderboard.sh)
↓
End

3 Features Implemented

- **Authentication System** :
  - Supports user registration and login with passwords securely hashed using SHA-256.

  - Two-player authentication is required before starting a game.
  - The system enforces a 3-attempt login limit per user, after which a temporary time-based lockout is applied.

- **Blitz Feature** :
  - A timed gameplay mode where each player must make a move within a specified time limit.
  - The remaining time is visually represented using a vertical timer bar which changes colour according to the urgency of move.

- **Animations** : Visual enhancements implemented to improve user experience:
  - Tic Tac Toe: drawing animations for X and O symbols, along with a line animation highlighting the winning combination.
  - Othello: coin flipping with a wave-like animation to represent piece conversion visually.
  - Connect Four: a falling animation for coins to simulate realistic gravity-based placement.

- **Abstractions** :
  - **Developer Point of View**
    * Design elements which are inheritable as a Class Button, Box, Jagged Line etc.
    * Menus - callable as a function, returns the choice
  - **User Point of View**
    * GameHub folder is clean - system files and user files are hidden
    * Intuitive Menus and usage

4 Directory Structure

```
hub/
├── main.sh
├── game.py
├── leaderboard.sh
├── reset.sh
├── games/
│   ├── tictactoe.py
│   ├── connect4.py
│   ├── othello.py
│   ├── classGame.py
│   └── design_elements.py
├── images/
│   ├── red_disc.png
│   └── yellow_disc.png
└── .user_files/
```

```
│   ├── .stats_tictactoe.csv
│   ├── .stats_connect4.csv
│   ├── .stats_othello.csv
│   ├── .users.tsv
│   ├── .user_total_wins.csv
│   ├── .top5.txt
│   └── .game_frequencies.csv
└── .system_files/
    └── .failed_logins.csv
```

5 Installation & Setup

```
pip install numpy pygame matplotlib
```

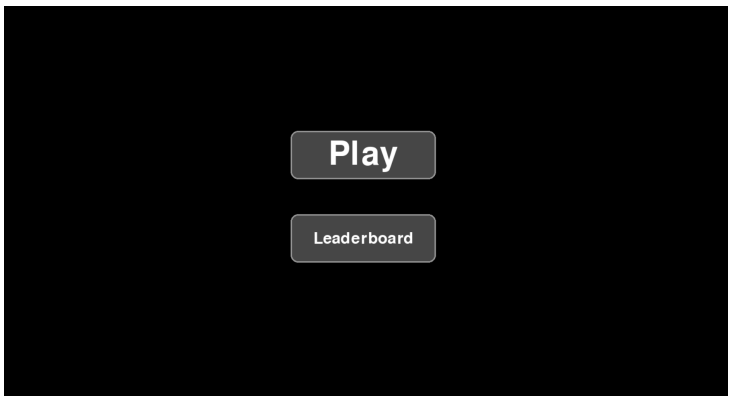
6 Usage

Run the main script to start the application:

```
bash main.sh
```

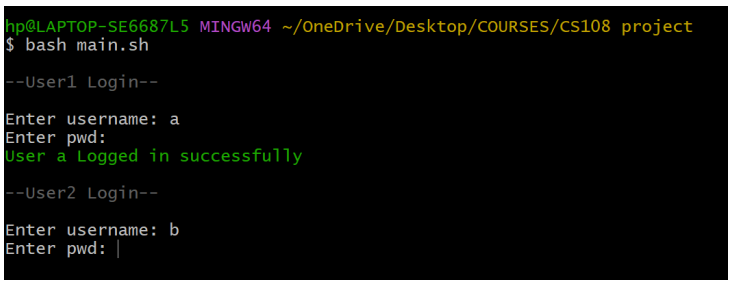
Main Menu

After launching, the main menu is displayed with options to start a game or view the leaderboard.



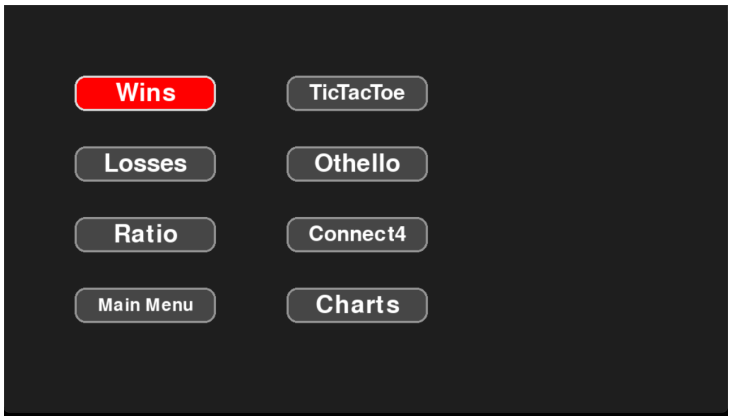
Authentication

Two users are prompted to log in via the terminal. Each user has up to three attempts before a temporary lockout.



Leaderboard Menu

Users can view statistics sorted by wins, losses, or win/loss ratio, and also filter by game.



Leaderboard Output

The leaderboard is displayed in the terminal in a formatted table.

7 Retrospective

- Accidental change of the same part by both teammates -
  Fix: Clearly communicating beforehand, and also doing a merge commit when necessary

- Lagging animation -
  Fix: Performance improved by precomputing static frames onto Surface objects and then blitting them to the screen at loop runtime [7]

- Timer and Toss Bar fluid going out of box for low levels -
  Fix: Using pygame Surfaces transparency feature - made a window Surface which lets viewer peek through only the shape we want, hence clipping the fluid [1]

- Scaling of sizes across systems -
  Fix: Using scaling functions wherever necessary

- Not able to change game board values in Othello - the diagonal array extracted from game board using the diagonal() function was not writable by default.
  Fix: Used arr_name.setflags(write=1) [2]

- Seed was same for all board lines since time was converted to int, and execution was finished in less than a second.
  Fix: Used int(time_ns())
- Unnecessary complicated scaling by introducing a Size class -
  Fix: Removed overcomplicated Size class, changed it to simple scaling functions
- Pygame Events during toss() like random clicks and ESCAPE were getting transferred to next loop -
  Fix: `pygame.event.clear()`

8 What we would do differently with 10 more hours

- **Code Architecture Improvements**
  - Introduce a state-machine type flow for game.py to simplify the logic
  - Create a Menu Class with Button and Box as subclass
  - Use enums for things like whose turn is it to avoid all the confusion during development
- **User Experience Enhancements**
  - Add smoother scene transitions (fade in/out between menus and games).
  - Improve aesthetics and feedback (hover effects, button animations, sound effects).
  - Add a pause button
- **Feature Expansion**
  - Add single-player mode which just chooses random choices (Just for fun)
  - Figure out a cleaner leaderboard display aesthetic so we can see terminal output simultaneously

References

[1] *How to Draw a Semi-Transparent Circle in Pygame.* URL: https://stackoverflow.com/questions/51229740/how-to-draw-a-semi-transparent-circle-in-pygame.

[2] *How to Make a NumPy Array Read-Only.* URL: https://www.geeksforgeeks.org/python/how-to-make-a-numpy-array-read-only/.

[3] *Hyperlinks in LaTeX.* URL: https://www.overleaf.com/learn/latex/Hyperlinks.

[4] *Making a Simple Directory Tree.* URL: https://tex.stackexchange.com/questions/5073/making-a-simple-directory-tree.

[5] *NumPy Absolute Beginners Guide.* URL: https://numpy.org/doc/stable/user/absolute_beginners.html.

[6] *Pygame Draw Module Documentation*. URL: https://www.pygame.org/docs/ref/draw.html.

[7] *Pygame Surface Documentation*. URL: https://www.pygame.org/docs/ref/surface.html#pygame.Surface.

[8] *Pygame Transform Smoothscale*. URL: https://www.pygame.org/docs/ref/transform.html#pygame.transform.smoothscale.

[9] *Spines Demo*. URL: https://matplotlib.org/stable/gallery/spines/spines.html.

[10] *Subplots Demo*. URL: https://matplotlib.org/stable/gallery/subplots_axes_and_figures/subplots_demo.html.